

EXPERIMENT - 3

Title

Employee-Department-Project Schema and SQL Queries

Aim

To create an Employee-Department-Project database schema, insert at least 30 employees across 5 departments and 8 projects, and perform SQL queries using selection, projection, aggregate functions, GROUP BY, HAVING, CASE expressions, and ORDER BY.

Objectives

1. To create Employee, Department, and Project tables.
2. To insert 30 employee records, 5 department records, and 8 project records.
3. To perform selection and projection operations.
4. To use aggregate functions such as COUNT(), SUM(), AVG(), MIN(), and MAX().
5. To use GROUP BY and HAVING clauses.
6. To use CASE expressions for conditional classification.
7. To sort records using ORDER BY.

Theory

A relational database is a system used to store and manage data in the form of tables. A table consists of rows and columns, where rows represent individual records and columns represent attributes of the records.

In this experiment, three tables are created: **Employee**, **Department**, and **Project**. The **Department** table stores information about different departments such as department ID and department name. The **Project** table stores information related to different projects. The **Employee** table stores employee details such as employee ID, employee name, salary, department ID, and project ID.

The Employee table is connected with the Department and Project tables using **foreign keys**. These relationships help maintain consistency between the tables and allow data from multiple tables to be accessed using SQL queries.

1. Selection

Selection is the process of retrieving only those rows from a table that satisfy a particular condition. In SQL, selection is generally performed using the **WHERE** clause.

For example, if we want to display only employees whose salary is greater than 50,000, we can use a condition with the WHERE clause.

Purpose of Selection:

- To filter records according to a condition.
- To retrieve only the required rows.
- To make the output more specific and useful.

2. Projection

Projection means selecting only the required columns from a table. It is performed using the **SELECT** statement in SQL.

For example, instead of displaying all details of employees, we can display only the employee name and salary.

Purpose of Projection:

- To display only required columns.
- To reduce unnecessary information in the output.
- To make the result easier to understand.

3. Aggregate Functions

Aggregate functions are built-in SQL functions used to perform calculations on a group of records and return a single result. They are useful for calculating totals, averages, minimum values, maximum values, and the number of records.

The commonly used aggregate functions are:

- **COUNTO** - Counts the number of records or values.
- **SUMO** - Calculates the total of numeric values.
- **AVGO**- Calculates the average of numeric values.
- **MINO** - Finds the smallest value.
- **MAXO**- Finds the largest value.

For example, aggregate functions can be used to find the total salary of employees, average salary, highest salary, lowest salary, or total number of employees.

4. GROUP BY

The **GROUP BY** clause is used to divide records into groups based on one or more columns. It is generally used along with aggregate functions.

For example, employees can be grouped according to their department. We can then calculate the number of employees or average salary for each department.

Purpose of GROUP BY:

- To create groups of similar records.
- To perform calculations for each group.
- To generate summarized results.

5. HAVING

The **HAVING** clause is used to apply conditions to groups created using the **GROUP BY** clause. It is mainly used with aggregate functions.

The **WHERE** clause filters individual rows before grouping, whereas **HAVING** filters the groups after the grouping operation.

For example, if we want to display only those departments whose average employee salary is greater than 50,000, the **HAVING** clause can be used.

6. CASE

The **CASE** expression is used to perform conditional operations in SQL. It works similar to an **IF-ELSE** statement in programming languages.

It checks a condition and returns a particular value when the condition is true. Different results can be returned for different conditions.

For example, employee salaries can be classified as:

- High Salary
- Medium Salary
- Low Salary

This helps in presenting data in a meaningful and understandable form.

7. ORDER BY

The **ORDER BY** clause is used to arrange the records in a particular order. The result can be arranged in **ascending order (ASC)** or **descending order (DESC)**.

By default, SQL generally uses ascending order if no order is specified.

For example, employee salaries can be arranged from lowest to highest using ASC or from highest to lowest using DESC.

Purpose of ORDER BY:

- To sort the query result.
- To arrange records in ascending or descending order.

SQL Implementation

1. Create Database

```
CREATE DATABASE CompanyDB;  
USE CompanyDB;
```

Output :

Note: In ByteXL/Nimbus, the database is already provided as sandbox_db, so these commands are **not executed there**. The tables are created directly inside the provided database.

2. Create Department Table

```
CREATE TABLE Department (  
    DeptID INT PRIMARY KEY,  
  
    DeptName VARCHAR(50) NOT NULL UNIQUE );
```



3. Create Project Table

```
CREATE TABLE Project (
    ProjectID INT PRIMARY KEY,
    ProjectName VARCHAR(100) NOT NULL,
    Budget DECIMAL(12,2) NOT NULL
);
```



4. Create Employee Table

```
CREATE TABLE Employee (
    EmpID INT PRIMARY KEY,
    EmpName VARCHAR(50) NOT NULL,
    Salary DECIMAL(10,2) NOT NULL,
    DeptID INT,
    ProjectID INT,
    FOREIGN KEY (DeptID) REFERENCES Department(DeptID),
    FOREIGN KEY (ProjectID) REFERENCES Project(ProjectID) );
```

5.1 Insert Department Data

```
INSERT INTO Department (DeptID, DeptName) VALUES
```

```
(101, 'Computer Science'),
```

```
Aryan Pratap Singh
```

```
25LBCS3266
```

(102, 'Human Resources'),
(103, 'Finance'),
(104, 'Marketing'),
(105, 'Research and Development');

5.2 Insert Project Data

INSERT INTO Project (ProjectID, ProjectName, Budget) VALUES
(201, 'AI Chatbot', 800000),
(202, 'E-Commerce Platform', 1200000),
(203, 'Payroll Automation', 500000),
(204, 'Cyber Security System', 950000),
(205, 'Mobile Application', 700000),
(206, 'Data Analytics', 900000),
(207, 'Cloud Migration', 1100000),
(208, 'Research Portal', 650000);

5.3 Insert Employee Data

INSERT INTO Employee (EmpID, EmpName, Salary, DeptID, ProjectID) VALUES
(1, 'Aarav Sharma', 72000, 101, 201),
(2, 'Priya Singh', 65000, 101, 202),
(3, 'Rohan Verma', 58000, 101, 204),
(4, 'Ananya Gupta', 61000, 101, 206),
(5, 'Kunal Mehta', 55000, 101, 207),
(6, 'Neha Kapoor', 48000, 102, 203),
(7, 'Rahul Yadav', 52000, 102, 203),
(8, 'Sneha Jain', 60000, 102, 205),
(9, 'Amit Kumar', 45000, 102, 208),
(10, 'Ishita Roy', 57000, 102, 201),

(11,'Vikas Gupta',68000,103,206),
(12,'Simran Kaur',62000,103,202),
(13,'Aditya Mishra',59000,103,207),
(14,'Pooja Agarwal',51000,103,203),
(15,'Mohit Tiwari',73000,103,204),
(16,'Nisha Sharma',56000,104,205),
(17,'Arjun Singh',49000,104,201),
(18,'Riya Malhotra',63000,104,202),
(19,'Varun Saxena',54000,104,208),
(20,'Tanya Bansal',67000,104,206),
(21,'Dev Patel',75000,105,204),
(22,'Meera Joshi',66000,105,206),
(23,'Sahil Khan',58000,105,207),
(24,'Kavya Nair',70000,105,208),
(25,'Manish Jain',53000,105,201),
(26,'Isha Verma',46000,101,205),
(27,'Yash Thakur',50000,102,207),
(28,'Aditi Sinha',64000,103,208),
(29,'Harsh Vardhan',57000,104,204),
(30,'Diya Agarwal',69000,105,202);

Outputs:

f- → C → app.bytext.ai/nimbus/454rqctf2

* 0 .



EXPLORER

Query1

Query2

Query1

select * from departments;

Search...

TABLER

department

departments

employee

employees

leaves

project

salaries

FUNCTIONS

@ GetAverageSalaryOfDepartment

department_id

101

102

103

104

101

department_name

Engineering

Analytics

Management

Marketing

app.bytext.ai/nimbus/454rqctf2

MySQL db_454rqctf2

Query2

all,...

select * from project;

Search...

department

department_id

employee_id

employee_name

leaves

project

salaries

FUNCTIONS

@ GetAverageSalaryOfDepartment

G) Appmvt_Ord_jectl_...

ProjectID

201

202

203

204

206

207

208

ProjectName

AIChatbot

E-commerce Platform

Payroll Automation

Security System

Mobile Application

Data Analytics

Cloud Migration

Enterprise Portal

800000.00

1200000.00

500000.00

900000.00

700000.00

900000.00

1100000.00

650000.00

* 0 .

UEDlaD

Query4

Aryan Pratap Singh
25LBCS3266

MySQL • db_454nqctf2

69 SELECT *
70 FROM Employee
71 WHERE Salary > 60000;1

Search

TASUS
El department
El departments
e employee
e employees
El leavet
El p"QKt
El salaries
FUNaIONS
@ GetAverageSalaryOfDepartment
PROCEDURS
!!! ApproveOrRejectleave

EmpID
21
12
15
18
20
21
22
24
28
29

EmpName
Aarav Sharma
PriyaSingh
Ananya Gupta
VikasGupta
Simran Kaur
MohitTiwari
RiyaMalhotra
Tanya Bansal
Dev Patel
MeeraJoshi
aNaair
Aditi Sinha
Pooja Agarwal

Salary
72000.00
65000.00
61000.00
68000.00
62000.00
73000.00
63000.00
67000.00
75000.00
66000.00
70000.00
64000.00
51000.00

DeptID
101
101
101
103
103
103
104
104
105
105
105
103
105

ProjectID
201
202
206
206
202
204
202
206
204
206
208
208
208

> c:\ SavedQueries(17)

6.2 Projection

SELECT EmpName, Salary
FROM Employee;

MySQL • db_454nqctf2

73 SELECT EmpName, salary
FROM Employee;

d1-p;Inmen1
d-rtmcub
B employee
B employees
B leavet
E project
E ,,,,in
GetAverageSalaryOfDepartment
@ ApproveOrRejectleave

EmpName
Aarav Sharma
PriyaSingh
Rohan Verma
Anshu Gupta
Kunal Mehta
Neha Kapoor
Rishul Yadav
Snehalini
Anshu Kumar
Vikas Gupta
Simran Kaur
Pooja Agarwal

Salary
72000.00
65000.00
58000.00
61000.00
55000.00
60000.00
52000.00
60000.00
45000.00
57000.00
60000.00
62000.00
59000.00
51000.00

> Q S.v.d\Quoin(17)

The screenshot shows a web-based MySQL IDE interface. The top bar displays the URL 'app.bytext.ai/nimbus/454rqctf2' and the database name 'MySQL - db_454rqctf2'. The left sidebar contains an 'EXPLORER' panel with a search bar and a list of tables: 'department', 'departments', 'employee', 'employees', 'leaves', 'project', 'salaries', and 'FUNCTIONS'. The 'departments' table is selected. The main area shows a SQL query: 'SELECT * FROM Employee WHERE Salary > 60000;'. Below the query, the results are displayed in a table with two columns: 'Employee' and 'Salary'.

Employee	Salary
Nisha Sharma	56000.00
Arjun Singh	49000.00
Riya Malhotra	63000.00
Varun Saxena	54000.00
Tanya Bansal	67000.00
Dev Patel	75000.00
Meera Joshi	66000.00
Sahil Khan	58000.00
Kavya Nair	70000.00
Manish Jain	53000.00
Isha Verma	46000.00
Yash Thakur	50000.00
Aditi Sinha	64000.00
Harsh Vardhan	57000.00

6.3 Aggregate Functions

SELECT COUNT(*) AS TotalEmployees,

SUM(Salary) AS TotalSalary,

AVG(Salary) AS AverageSalary,

MIN(Salary) AS MinimumSalary,

MAX(Salary) AS MaximumSalary

FROM Employee;

The screenshot shows a web-based MySQL IDE interface. The top bar displays the URL 'app.bytext.ai/nimbus/454rqctf2' and the database name 'MySQL - db_454rqctf2'. The left sidebar contains an 'EXPLORER' panel with a search bar and a list of tables: 'department', 'departments', 'employee', 'employees', 'leaves', 'project', 'salaries', and 'FUNCTIONS'. The 'departments' table is selected. The main area shows a SQL query: 'SELECT COUNT(*) AS TotalEmployees, SUM(Salary) AS TotalSalary, AVG(Salary) AS AverageSalary, MIN(Salary) AS MinimumSalary, MAX(Salary) AS MaximumSalary FROM Employee;'. Below the query, the results are displayed in a table with five columns: 'TotalEmployees', 'TotalSalary', 'AverageSalary', 'MinimumSalary', and 'MaximumSalary'.

TotalEmployees	TotalSalary	AverageSalary	MinimumSalary	MaximumSalary
30	1783000.00	59433.333333	45000.00	75000.00

6.4 GROUP BY

SELECT DeptID, COUNT(*) AS EmployeeCount

FROM Employee

GROUP BY DeptID;

Aryan Pratap Singh
25LBCS3266

+

-

→

C

!;

app.bytextl.ai/mmbus/4S4rqctf2

MySQL • db_454rqctf2

☆

📄

🔍

⌵

🔍

⚙️

AI

▶️ Run

83

SELECT deptID, COUNT(*) AS EmployeeCount

84

FROM Employee

85

GROUP BY DeptID;

Search...

🔍

department

🔍

dept,utments

🔍

employee

🔍

employees

🔍

leaves

🔍

project

🔍

salaries

FUNCTIONS

G, GelAverageSalaryOfDepartment

PROCEDURES

G, ApproveOrRejectleave

OCptID

EmployeeCount

101

102

103

104

105

6.5 GROUP BY with AVG()

```
SELECT DeptID, AVG(Salary) AS AverageSalary
FROM Employee
GROUP BY DeptID;
```

MySQL • db_454rqctf2

🔍

⚙️

AI

▶️ Run

87

SELECT DeptID, AVG(Salary) AS AverageSalary

88

FROM Employee

89

GROUP BY deptID;

Search...

🔍

department

🔍

dept,utments

🔍

employee

🔍

employees

🔍

leaves

🔍

project

🔍

salaries

FUNCTIONS

G, GelAverageSalaryOfDepartment

PROCEDURES

G, ApproveOrRejectleave

DeptID

AverageSalary

101

59500.000000

102

52000.000000

103

62833.333333

104

57666.666667

105

65166.666667

6.6 HAVING

```
SELECT DeptID, AVG(Salary) AS AverageSalary
FROM Employee
GROUP BY DeptID
HAVING AVG(Salary) > 60000;
```

MySQL • db_454rqctf2

☆

0

•

1111:DEI:!!

91

SELECT DeptID, AVG(Salary) AS AverageSalary

92

FROM Employee

93

GROUP BY DeptID

94

HAVING AVG(Salary) > 60000;

Search...

TABLES

🔍

department

🔍

dept,utments

🔍

employee

🔍

employees

🔍

leaves

🔍

project

🔍

salaries

OCptID

AverageSalary

103

62833.333333

105

65166.666667

Aryan Pratap Singh
25LBCS3266

6.7 CASE Expression

SELECT EmpName, Salary,

CASE

WHEN Salary>= 65000 THEN 'High'

WHEN Salary>= 55000 THEN 'Medium'

ELSE 'Low'

END AS SalaryCategory

FROM Employee;

Search...

E) department

B) dep111ments

B) employee

E) employtS

E) leaves

E) project

E) salari

ti) GetAveragesala,yOfOepanmen1

@ ApproveOrRejeedleave

> [J S11ved Querit (17)

MySQL- db_454fqctf2

96 SELECT EmpName, Salary,

97 CASE

98 WHEN salary >= 65000 THEN 'High'

99 WHEN Salary >= 55800 THEN 'Medium'

100 ELSE 'Low'

101 END AS SalaryCategory

102 FROM Employee;

EmpName	Salary	SalaryCategory
AartiN Sharma	72000.00	High
Priya Singh	65000.00	High
Rohan Verma	58000.00	Medium
Ananya Gupta	61000.00	Medium
Kunal Mehta	55000.00	Medium
Neha Kapoor	48000.00	
Rahul Yadav	52000.00	
Snehalain	60000.00	Medium
Amit Kumar	45000.00	
Ishita Roy	57000.00	Medium
Vikas Gupta	68000.00	High
Simran Kaur	62000.00	Medium
Aditya Mishra	59000.00	Medium
Pooja Agarwal	51000.00	

MySQL- db_54rqctf2

% SELECT EmpName, Salary,

98 CASE

99 WHEN salary >= 65000 THEN 'High'

100 WHEN Salary >= 55800 THEN 'Medium'

101 ELSE 'Low'

102 END AS SalaryCategory

103 FROM Employee;

EmpName	Salary	SalaryCategory
Nisha Sharma	56000.00	Medium
ArjunSingh	49000.00	
Riya Malhotra	63000.00	Medium
Tanya Bansal	54000.00	
Dev Patel	67000.00	High
Meeraj Joshi	75000.00	High
Ishita Nair	58000.00	Medium
MarishJalri	70000.00	High
Ishita Verma	53000.00	
Yash Thakur	46000.00	
Aditya Sinhal	50000.00	
Harsh Vardhan	64000.00	Medium
Diya Agarwal	57000.00	Medium
	69000.00	High

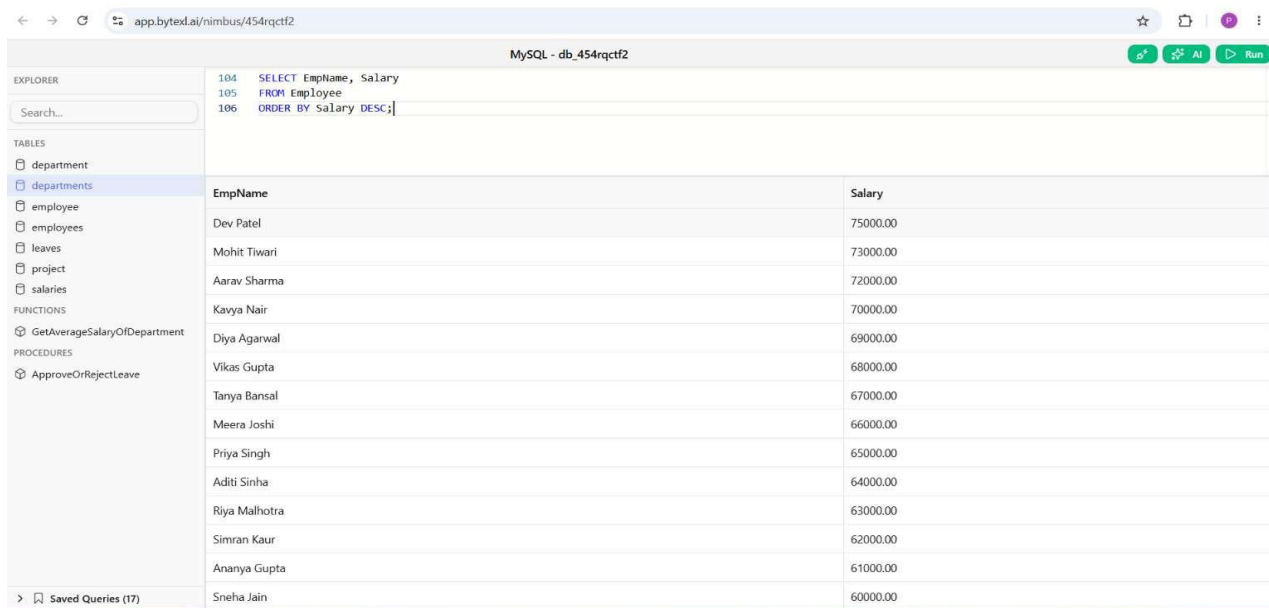
Aryan Pratap Singh
25LBCS3266

6.8 ORDER BY

SELECT EmpName, Salary

FROM Employee

ORDER BY Salary DESC;

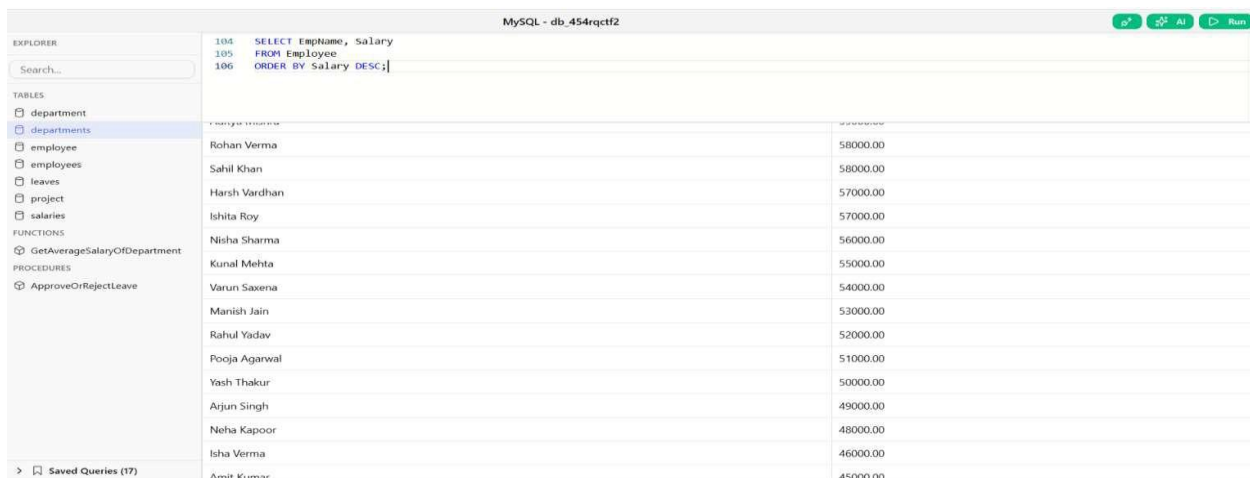


The screenshot shows a MySQL database interface with a query editor on the left and a results table on the right. The query editor contains the following SQL code:

```
104 SELECT EmpName, Salary
105 FROM Employee
106 ORDER BY Salary DESC;
```

The results table displays the following data:

EmpName	Salary
Dev Patel	75000.00
Mohit Tiwari	73000.00
Aarav Sharma	72000.00
Kavya Nair	70000.00
Diya Agarwal	69000.00
Vikas Gupta	68000.00
Tanya Bansal	67000.00
Meera Joshi	66000.00
Priya Singh	65000.00
Aditi Sinha	64000.00
Riya Malhotra	63000.00
Simran Kaur	62000.00
Ananya Gupta	61000.00
Sneha Jain	60000.00



The screenshot shows a MySQL database interface with a query editor on the left and a results table on the right. The query editor contains the following SQL code:

```
104 SELECT EmpName, salary
105 FROM Employee
106 ORDER BY Salary DESC;
```

The results table displays the following data:

EmpName	salary
Rohan Verma	58000.00
Sahil Khan	58000.00
Harsh Vardhan	57000.00
Ishita Roy	57000.00
Nisha Sharma	56000.00
Kunal Mehta	55000.00
Varun Saxena	54000.00
Manish Jain	53000.00
Rahul Yadav	52000.00
Pooja Agarwal	51000.00
Yash Thakur	50000.00
Arjun Singh	49000.00
Neha Kapoor	48000.00
Isha Verma	46000.00
Amit Kumar	45000.00

Result

The Employee-Department-Project database schema was successfully created. Five departments, eight projects, and thirty employees were successfully inserted. SQL queries demonstrating selection, projection, aggregate functions, GROUP BY, HAVING, CASE expressions, and ORDER BY were successfully executed.

Aryan Pratap Singh
25LBCS3266

Learning Outcomes

After completing this experiment, we are able to:

1. Create tables in a relational database.
2. Define primary keys and foreign keys.
3. Insert and retrieve records.
4. Perform selection and projection operations.
5. Use aggregate functions for data analysis.
6. Group records using GROUP BY.
7. Filter grouped data using HAVING.
8. Use CASE expressions for conditional classification.
9. Sort records using ORDER BY.

Conclusion

This experiment provided practical knowledge of relational database design and SQL operations. The Employee-Department-Project schema was successfully implemented, and different SQL queries were used to retrieve, calculate, group, filter, classify, and sort data.